

Coleções em Java: ArrayList, HashSet e HashMap

Parte 1: Por que precisamos de Coleções?

O problema: arrays têm tamanho fixo

Até agora, para guardar vários valores usamos **arrays**. Mas um array tem um problema sério: o tamanho é definido na criação e **nunca muda**.

```
1 String[] alunos = new String[3];
2 alunos[0] = "Maria";
3 alunos[1] = "João";
4 alunos[2] = "Ana";
5
6 // E se chegar um quarto aluno? Não há espaço!
7 // alunos[3] = "Pedro"; // ArrayIndexOutOfBoundsException
8
9 // A unica saída seria criar um array NOVO e maior, e copiar tudo:
10 String[] alunosNovo = new String[4];
11 for (int i = 0; i < alunos.length; i++) {
12     alunosNovo[i] = alunos[i];
13 }
14 alunosNovo[3] = "Pedro"; // trabalhoso e repetitivo!
```

Listing 1: O problema do array de tamanho fixo

Outras limitações dos arrays

- Não crescem nem encolhem automaticamente.
- Não têm métodos prontos para adicionar, remover ou verificar se um elemento existe – é preciso programar tudo manualmente.
- Permitem facilmente valores duplicados, mesmo quando isso não faz sentido (duas vezes o mesmo CPF, por exemplo).
- Não existe uma forma nativa de associar uma **chave** a um **valor** (como “nome do produto” → “preço”).

O que é a Java Collections Framework?

Definição: Coleção

Uma **coleção** (*collection*) é um objeto que agrupa vários elementos e oferece métodos prontos para **adicionar**, **remover**, **buscar** e **percorrer** esses elementos – crescendo e encolhendo automaticamente, conforme a necessidade. Em Java, essas estruturas ficam no pacote `java.util`.

Analogia: gavetas organizadoras

Pense em três tipos de gaveta organizadora:

- Uma **lista numerada**, onde cada item fica em uma posição e pode se repetir (a fila do banco: posição 1, 2, 3...).
- Um **conjunto**, onde cada item só pode aparecer **uma vez** (a lista de CPFs cadastrados: não existe CPF repetido).
- Um **dicionário**, onde cada palavra (chave) aponta para seu significado (valor) – como uma agenda de contatos: nome → telefone.

Em Java, essas três ideias viram, respectivamente, `ArrayList`, `HashSet` e `HashMap`.

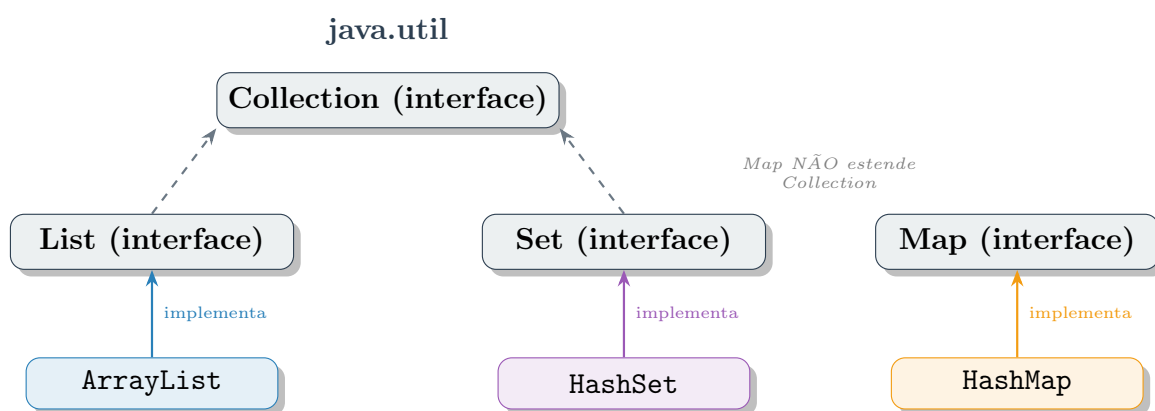


Figure 1: Hierarquia simplificada: `List` e `Set` são tipos de `Collection`; `Map` é uma interface independente, pois trabalha com pares chave-valor.

Estrutura	Permite duplicados?	Mantém ordem de inserção?	Como guarda os dados
<code>ArrayList</code>	Sim	Sim	Lista indexada (posição 0, 1, 2...)
<code>HashSet</code>	Não	Não garante	Conjunto sem repetição
<code>HashMap</code>	Chaves não, valores sim	Não garante	Pares chave → valor

Importações necessárias

Para usar essas estruturas, é preciso importar do pacote `java.util`:

```
1 import java.util.ArrayList;
2 import java.util.HashSet;
3 import java.util.HashMap;
```

Parte 2: ArrayList

O que é um ArrayList?

Definição: ArrayList

`ArrayList` é uma **lista dinâmica**: funciona como um array, mas **cresce e encolhe automaticamente** conforme elementos são adicionados ou removidos. Cada elemento ocupa uma **posição (índice)**, começando em 0, e **permite valores repetidos**.

Analogia: o trem de vagões expansível

Pense em um trem onde você pode **encaixar um vagão novo** sempre que quiser, no final ou em qualquer posição. Também pode **desengatar** um vagão do meio, e os vagões seguintes deslizam para preencher o espaço. Não existe um número fixo de vagões – o trem cresce e encolhe sob demanda.

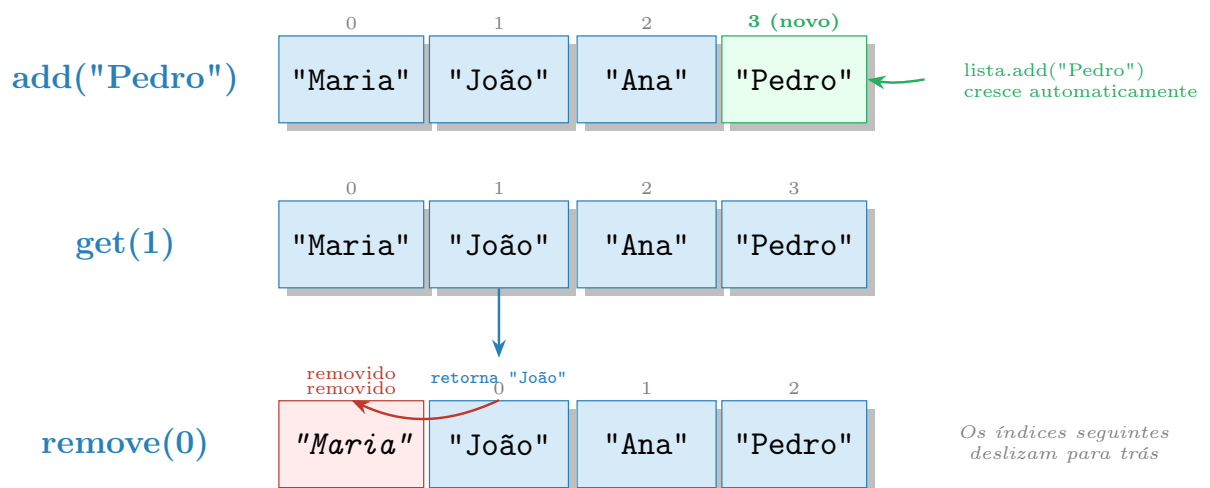


Figure 2: Um `ArrayList` cresce ao adicionar elementos, permite buscar por índice com `get()`, e reorganiza os índices automaticamente ao remover um elemento.

Sintaxe básica

```
1 import java.util.ArrayList;
2
```

```

3 public class Main {
4     public static void main(String[] args) {
5
6         // Cria uma lista de Strings (vazia)
7         ArrayList<String> alunos = new ArrayList<>();
8
9         // Adiciona elementos no final
10        alunos.add("Maria");
11        alunos.add("João");
12        alunos.add("Ana");
13
14        System.out.println(alunos);           // [Maria, João, Ana]
15        System.out.println(alunos.size());    // 3
16        System.out.println(alunos.get(1));    // João
17
18        alunos.remove("João");                // remove pelo VALOR
19        alunos.remove(0);                     // remove pela POSIÇÃO
20
21        System.out.println(alunos);           // [Ana]
22    }
23 }

```

Listing 2: Criando e usando um ArrayList

Atenção ao tipo dentro de <>

Um ArrayList só guarda **objetos**, não tipos primitivos. Para guardar números, use as classes **wrapper**: int → Integer, double → Double, boolean → Boolean.

```

1 ArrayList<int> numeros = new ArrayList<>();    // ERRO de
   compilação!
2 ArrayList<Integer> numeros = new ArrayList<>(); // correto
3 numeros.add(10);    // o Java converte automaticamente (autoboxing
   )

```

Principais métodos do ArrayList

Método	O que faz
<code>add(valor)</code>	Adiciona um elemento no final da lista
<code>add(indice, valor)</code>	Insere o elemento em uma posição específica
<code>get(indice)</code>	Retorna o elemento daquela posição
<code>set(indice, valor)</code>	Substitui o elemento daquela posição
<code>remove(indice)</code>	Remove o elemento daquela posição
<code>remove(valor)</code>	Remove a primeira ocorrência daquele valor
<code>contains(valor)</code>	Retorna <code>true</code> se o valor existe na lista
<code>indexOf(valor)</code>	Retorna a posição do valor (ou -1 se não existir)
<code>size()</code>	Retorna a quantidade de elementos
<code>isEmpty()</code>	Retorna <code>true</code> se a lista está vazia
<code>clear()</code>	Remove todos os elementos

Percorrendo um ArrayList

```
1 ArrayList<String> frutas = new ArrayList<>();
2 frutas.add("Maçã");
3 frutas.add("Banana");
4 frutas.add("Laranja");
5
6 // 1) for tradicional, usando indice e get()
7 for (int i = 0; i < frutas.size(); i++) {
8     System.out.println(frutas.get(i));
9 }
10
11 // 2) for-each: mais simples, não precisa de indice
12 for (String fruta : frutas) {
13     System.out.println(fruta);
14 }
15
16 // 3) imprimindo a lista inteira de uma vez
17 System.out.println(frutas); // [Maçã, Banana, Laranja]
```

Listing 3: Três formas de percorrer um ArrayList

Exemplo completo: lista de tarefas

```

1 import java.util.ArrayList;
2
3 public class ListaDeTarefas {
4     public static void main(String[] args) {
5
6         ArrayList<String> tarefas = new ArrayList<>();
7
8         tarefas.add("Estudar Java");
9         tarefas.add("Lavar a louça");
10        tarefas.add("Fazer exercício");
11        tarefas.add("Ler um livro");
12
13        System.out.println("--- Tarefas pendentes ---");
14        for (int i = 0; i < tarefas.size(); i++) {
15            System.out.println((i + 1) + ". " + tarefas.get(i));
16        }
17
18        // Concluiu a tarefa "Lavar a louça"
19        tarefas.remove("Lavar a louça");
20
21        System.out.println("\n--- Após concluir uma tarefa ---");
22        for (String tarefa : tarefas) {
23            System.out.println("- " + tarefa);
24        }
25
26        System.out.println("\nTarefas restantes: " + tarefas.size()
27        );
28        System.out.println("Contém 'Ler um livro'? " + tarefas.
29        contains("Ler um livro"));
30    }
31 }

```

Listing 4: Sistema simples de lista de tarefas

Saída esperada

```

--- Tarefas pendentes ---
1. Estudar Java
2. Lavar a louça
3. Fazer exercício
4. Ler um livro

--- Após concluir uma tarefa ---
- Estudar Java
- Fazer exercício
- Ler um livro

Tarefas restantes: 3
Contém 'Ler um livro'? true

```

Erros comuns com ArrayList

1. **Acessar um índice que não existe.** `frutas.get(10)` em uma lista de 3 elementos lança `IndexOutOfBoundsException`.
2. **Confundir `remove(int)` com `remove(Object)`.** Em uma `ArrayList<Integer>`, `remove(1)` remove o elemento da **posição** 1, enquanto `remove(Integer.valueOf(1))` remove o **valor** 1.
3. **Modificar a lista enquanto percorre com `for-each`.** Adicionar ou remover elementos dentro de um `for (String f : frutas)` causa `ConcurrentModificationException`.
4. **Esquecer que índices começam em 0.** O primeiro elemento é `get(0)`, não `get(1)`.

Verifique seu entendimento

Em uma `ArrayList<String>` com os elementos `["A", "B", "C", "D"]`, qual é o conteúdo da lista após `lista.remove(1)`? E qual índice o elemento "C" passa a ocupar?

Parte 3: HashSet

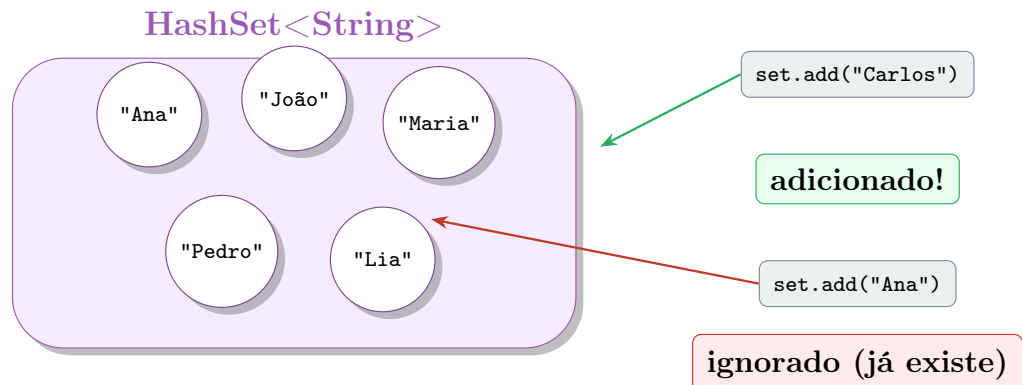
O que é um HashSet?

Definição: HashSet

`HashSet` é uma coleção que representa um **conjunto**: não permite **elementos duplicados** e **não garante nenhuma ordem** entre os elementos. Ele não tem índices – não existe `get(i)` em um `HashSet`.

Analogia: a lista de convidados

Imagine a lista de convidados de uma festa. Não faz sentido o mesmo nome aparecer duas vezes – a pessoa já está convidada, ponto final. Quando alguém tenta se cadastrar de novo, o sistema simplesmente **ignora** a tentativa repetida. É exatamente assim que o `HashSet` se comporta com `add()`.



Sem posições fixas – a ordem de impressão pode variar.

Figure 3: Um `HashSet` aceita elementos novos, mas ignora silenciosamente uma tentativa de adicionar um valor que já existe.

Sintaxe básica

```

1 import java.util.HashSet;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         HashSet<String> participantes = new HashSet<>();
7
8         participantes.add("Ana");
9         participantes.add("João");
10        participantes.add("Ana"); // tentativa duplicada: ignorada
11
12        System.out.println(participantes);           // [Ana, João] (
13        ordem pode variar)
14        System.out.println(participantes.size());    // 2 -- nao 3!
15
16        System.out.println(participantes.contains("João")); // true
17        participantes.remove("João");
18        System.out.println(participantes);           // [Ana]
19    }
20 }
```

Listing 5: Criando e usando um `HashSet`

Como o `HashSet` sabe que é duplicado?

O `HashSet` usa os métodos `equals()` e `hashCode()` de cada objeto para decidir se dois elementos são “iguais”. Para tipos prontos do Java, como `String` e `Integer`, isso já funciona automaticamente: duas `String` com o mesmo texto são consideradas o mesmo elemento, mesmo sendo objetos diferentes na memória.

Principais métodos do HashSet

Método	O que faz
<code>add(valor)</code>	Adiciona o elemento (ignora se já existir)
<code>remove(valor)</code>	Remove o elemento, se existir
<code>contains(valor)</code>	Retorna <code>true</code> se o elemento existe no conjunto
<code>size()</code>	Retorna a quantidade de elementos
<code>isEmpty()</code>	Retorna <code>true</code> se o conjunto está vazio
<code>clear()</code>	Remove todos os elementos

O que NÃO existe em um HashSet

- **Não existe `get(indice)`:** como não há posições fixas, não é possível pedir “o elemento da posição 2”.
- **Não existe ordem garantida:** imprimir o mesmo `HashSet` em execuções diferentes pode mostrar os elementos em ordens diferentes.
- **Não há duplicatas:** `add()` de um valor repetido apenas retorna `false` e não altera o conjunto.

Se a **ordem de inserção** importa, mas duplicatas ainda devem ser proibidas, existe a alternativa `LinkedHashSet` (mesma ideia, mas preserva a ordem).

Exemplo completo: removendo duplicatas de uma lista

Um uso muito comum do `HashSet` é **eliminar valores repetidos** de uma coleção de dados, como uma lista de e-mails cadastrados.

```
1 import java.util.ArrayList;
2 import java.util.HashSet;
3
4 public class NomesUnicos {
5     public static void main(String[] args) {
6
7         ArrayList<String> respostas = new ArrayList<>();
8         respostas.add("Maria");
9         respostas.add("João");
10        respostas.add("Maria"); // repetido
11        respostas.add("Ana");
12        respostas.add("João"); // repetido
13        respostas.add("Pedro");
14
15        System.out.println("Total de respostas: " + respostas.size
16        ()); // 6
```

```

17     HashSet<String> unicos = new HashSet<>(respostas);
18
19     System.out.println("Participantes únicos: " + unicos.size()
20 ); // 4
21     System.out.println(unicos);
22 }

```

Listing 6: Usando HashSet para encontrar nomes únicos

Saída esperada

```

Total de respostas: 6
Participantes únicos: 4
[Ana, Maria, João, Pedro]   (a ordem pode variar na sua máquina)

```

Verifique seu entendimento

Se você tentar executar `conjunto.add("Ana")` duas vezes seguidas em um `HashSet` vazio, qual será o valor de `conjunto.size()` ao final? O segundo `add()` gera algum erro?

Parte 4: HashMap

O que é um HashMap?

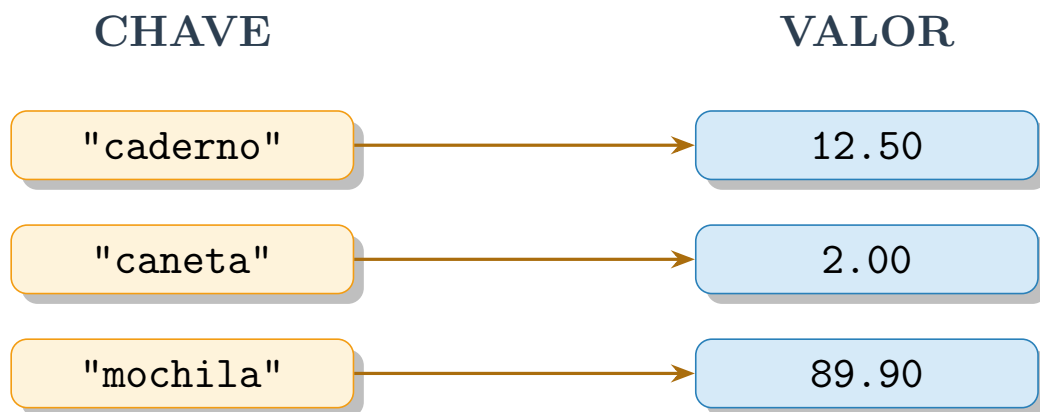
Definição: HashMap

`HashMap` é uma coleção que guarda dados em **pares chave-valor** (*key-value*). Cada **chave** é única – assim como em um `HashSet` – e está associada a exatamente um **valor**, que pode ser buscado rapidamente a partir da chave.

Analogia: a agenda de contatos

Em uma agenda telefônica, você procura um **nome** (a chave) e encontra o **telefone** correspondente (o valor). Não existem dois contatos com exatamente o mesmo nome de chave, mas dois nomes diferentes podem ter o mesmo número de telefone (dois valores iguais são permitidos).

HashMap<String, Double>



`map.get("caneta")` → 2.00

*Cada chave aparece no máximo uma vez;
valores podem se repetir.*

Figure 4: Um HashMap associa cada chave a um único valor. A busca pelo valor é feita a partir da chave, sem precisar percorrer toda a estrutura.

Sintaxe básica

```
1 import java.util.HashMap;
2
3 public class Main {
4     public static void main(String[] args) {
5
6         // HashMap<TipoDaChave, TipoDoValor>
7         HashMap<String, Double> precos = new HashMap<>();
8
9         precos.put("caderno", 12.50);
10        precos.put("caneta", 2.00);
11        precos.put("mochila", 89.90);
12
13        System.out.println(precos); // {caderno=12.5,
14        caneta=2.0, mochila=89.9}
15        System.out.println(precos.get("caneta")); // 2.0
16
17        precos.put("caneta", 2.50); // mesma chave: ATUALIZA o
18        valor
19        System.out.println(precos.get("caneta")); // 2.5
20
21        System.out.println(precos.containsKey("mochila")); // true
22        System.out.println(precos.size()); // 3
23
24        precos.remove("caderno");
25        System.out.println(precos); // {caneta=2.5, mochila=89.9}
```

```

24     }
25 }

```

Listing 7: Criando e usando um HashMap

Cuidado com chaves que não existem

Chamar `get()` com uma chave que não existe retorna `null`, e não um erro – mas usar esse `null` sem verificar pode causar uma `NullPointerException` mais adiante.

```

1 Double preco = precos.get("borracha"); // null (chave nao existe
2     )
3 // Forma segura, com valor padrao:
4 double precoSeguro = precos.getDefault("borracha", 0.0); //
5     0.0

```

Principais métodos do HashMap

Método	O que faz
<code>put(chave, valor)</code>	Associa o valor à chave (ou atualiza, se a chave já existir)
<code>get(chave)</code>	Retorna o valor associado à chave (ou <code>null</code>)
<code>getDefault(chave, padrao)</code>	Retorna o valor, ou um padrão se a chave não existir
<code>remove(chave)</code>	Remove o par chave-valor
<code>containsKey(chave)</code>	Retorna <code>true</code> se a chave existe
<code>containsValue(valor)</code>	Retorna <code>true</code> se o valor existe
<code>keySet()</code>	Retorna o conjunto de todas as chaves
<code>values()</code>	Retorna a coleção de todos os valores
<code>size()</code>	Retorna a quantidade de pares chave-valor

Percorrendo um HashMap

```

1 HashMap<String, Double> precos = new HashMap<>();
2 precos.put("caderno", 12.50);
3 precos.put("caneta", 2.00);
4
5 // 1) Percorrendo apenas as CHAVES
6 for (String chave : precos.keySet()) {
7     System.out.println(chave + " -> " + precos.get(chave));
8 }

```

```

9
10 // 2) Percorrendo apenas os VALORES
11 for (Double valor : precos.values()) {
12     System.out.println(valor);
13 }
14
15 // 3) Percorrendo os pares (chave, valor) -- forma mais eficiente
16 for (Map.Entry<String, Double> par : precos.entrySet()) {
17     System.out.println(par.getKey() + " custa R$ " + par.getValue()
18 );
19 }

```

Listing 8: Três formas de percorrer um HashMap

Não esqueça o import do Map.Entry

Para usar Map.Entry no terceiro exemplo acima, é preciso importar também a interface Map: import java.util.Map;

Exemplo completo: estoque de uma loja

```

1 import java.util.HashMap;
2 import java.util.Map;
3
4 public class Estoque {
5     public static void main(String[] args) {
6
7         HashMap<String, Integer> estoque = new HashMap<>();
8
9         estoque.put("Caderno", 50);
10        estoque.put("Caneta", 120);
11        estoque.put("Mochila", 8);
12
13        System.out.println("--- Estoque inicial ---");
14        for (Map.Entry<String, Integer> item : estoque.entrySet())
15        {
16            System.out.println(item.getKey() + ": " + item.getValue()
17            () + " unidades");
18        }
19
20        // Venda de 5 cadernos: busca o valor atual e atualiza
21        String produto = "Caderno";
22        int quantidadeAtual = estoque.get(produto);
23        estoque.put(produto, quantidadeAtual - 5);
24
25        // Chegou um produto novo
26        estoque.put("Lápis", 200);
27
28        System.out.println("\n--- Após movimentações ---");
29        for (Map.Entry<String, Integer> item : estoque.entrySet())
30        {

```

```

28         System.out.println(item.getKey() + ": " + item.getValue
29         () + " unidades");
30     }
31     System.out.println("\nTemos 'Borracha' no estoque? "
32     + estoque.containsKey("Borracha"));
33     System.out.println("Tipos de produto cadastrados: " +
34     estoque.size());
35 }

```

Listing 9: Sistema de estoque usando HashMap

Saída esperada (a ordem de impressão pode variar)

```

--- Estoque inicial ---
Caderno: 50 unidades
Caneta: 120 unidades
Mochila: 8 unidades

--- Após movimentações ---
Caderno: 45 unidades
Caneta: 120 unidades
Mochila: 8 unidades
Lápis: 200 unidades

Temos 'Borracha' no estoque? false
Tipos de produto cadastrados: 4

```

Erros comuns com HashMap

1. **Usar get() sem checar se a chave existe.** Se a chave não existir, o retorno é null, e usá-lo diretamente (por exemplo, em uma conta matemática) lança `NullPointerException`.
2. **Esquecer que put() com uma chave repetida SUBSTITUI o valor antigo,** em vez de criar uma nova entrada.
3. **Tentar usar índices.** `HashMap` não tem `get(0)`; a busca é sempre pela chave.
4. **Esperar uma ordem específica ao imprimir.** Assim como o `HashSet`, o `HashMap` não garante a ordem dos pares chave-valor.

Verifique seu entendimento

Se você executar `mapa.put("x", 10)` e, em seguida, `mapa.put("x", 20)`, qual será o valor retornado por `mapa.get("x")`? O `mapa.size()` muda entre as duas chamadas?

Parte 5: Comparando as três coleções

ArrayList	HashSet	HashMap
✓ Permite duplicados ✓ Mantém ordem de inserção ✓ Acesso por índice (get) ✗ Sem busca direta por chave Use quando: a ordem e a posição importam (fila, histórico, lista de itens)	✗ Não permite duplicados ✗ Não garante ordem ✗ Sem acesso por índice ✓ <code>contains()</code> muito rápido Use quando: precisa garantir que não haja repetição (CPFs, e-mails únicos)	✓ Chaves únicas, valores podem repetir ✗ Não garante ordem ✓ Busca rapidíssima pela chave ✗ Sem acesso por índice numérico Use quando: precisa associar um dado a outro (nome → telefone)

Figure 5: Cada coleção resolve um problema diferente: ordem e posição (**ArrayList**), unicidade (**HashSet**) ou associação chave-valor (**HashMap**).

	ArrayList	HashSet	HashMap
Estrutura	Lista indexada	Conjunto	Pares chave-valor
Duplicados	Permitidos	Não permitidos	Chaves: não / Valores: sim
Ordem	Mantém inserção	Não garante	Não garante
Acesso	<code>get(indice)</code>	<code>contains(valor)</code>	<code>get(chave)</code>
Aceita null?	Sim, vários	Um único null	Uma chave null
Quando usar	Listas ordenadas, histórico	Evitar repetição	Associar dados

Como escolher a coleção certa?

Pergunte, nessa ordem:

1. Eu preciso associar um dado a outro (tipo dicionário)? Sim → **HashMap**.
2. A ordem dos elementos e a posição (índice) importam, e duplicatas são aceitáveis? Sim → **ArrayList**.
3. Só preciso garantir que não haja valores repetidos? Sim → **HashSet**.

Resumo geral dos conceitos

Conceito	Explicação simples	Pergunta-chave
ArrayList	Lista dinâmica com índices; aceita duplicados.	Preciso de ordem e posição?
HashSet	Conjunto sem elementos repetidos.	Preciso evitar duplicatas?
HashMap	Pares chave → valor, com chaves únicas.	Preciso associar um dado a outro?
<code>add()</code>	Adiciona um elemento (List/Set).	Como insiro um valor?
<code>put()</code>	Associa um valor a uma chave (Map).	Como insiro um par?
<code>get()</code>	Recupera um elemento por índice (List) ou por chave (Map).	Como leio um dado?
<code>contains()</code>	Verifica se um valor já existe (List/Set).	O dado já está aqui?
<code>entrySet()</code>	Permite percorrer os pares chave-valor de um Map.	Como percorro o mapa todo?

Exercícios

Antes de responder

As questões abaixo combinam teoria, leitura de código e implementação. Tente resolver no papel antes de testar no computador – isso ajuda a fixar o raciocínio sobre cada estrutura.

Questão 1. Conceitual. Explique, com suas palavras, a principal diferença entre um array comum (`String[] nomes = new String[5];`) e um `ArrayList<String>`. Cite pelo menos duas vantagens do `ArrayList`.

Questão 2. Conceitual. Por que o código abaixo imprime 2 em vez de 3? Explique o mecanismo que faz o `HashSet` reconhecer a repetição.

```
1 HashSet<String> nomes = new HashSet<>();
2 nomes.add("Carlos");
3 nomes.add("Bia");
4 nomes.add("Carlos");
5 System.out.println(nomes.size());
```

Questão 3. Leitura de código (ArrayList). Qual é a saída impressa pelo programa abaixo? Mostre o conteúdo da lista **depois de cada operação**.


```

1 ArrayList<Integer> numeros = new ArrayList<>();
2 numeros.add(10);
3 numeros.add(20);
4 numeros.add(30);
5 numeros.add(1, 99);
6 numeros.remove(0);
7 System.out.println(numeros);
8 System.out.println(numeros.get(1));

```

Questão 4. Leitura de código (HashMap). Qual é o valor final de saldo após a execução deste trecho? Explique o que aconteceu com a chave "João".

```

1 HashMap<String, Integer> saldos = new HashMap<>();
2 saldos.put("Maria", 100);
3 saldos.put("João", 50);
4 saldos.put("João", 200);
5 int saldo = saldos.get("João");

```

Questão 5. Depuração. O código a seguir compila, mas lança uma exceção em tempo de execução. Identifique a causa do erro e proponha uma correção usando `getOrDefault` ou `containsKey`.

```

1 HashMap<String, Integer> idades = new HashMap<>();
2 idades.put("Ana", 20);
3
4 int idadeDePedro = idades.get("Pedro");
5 System.out.println(idadeDePedro + 1);

```

Questão 6. Escolha da estrutura. Para cada situação abaixo, indique se a melhor escolha é `ArrayList`, `HashSet` ou `HashMap`, e justifique em uma frase:

- Guardar a sequência de jogadas de uma partida, na ordem em que aconteceram.
- Guardar os CPFs já cadastrados em um sistema, para impedir cadastro duplicado.
- Guardar o preço de cada produto de um supermercado, buscando pelo nome do produto.
- Guardar a lista de presença de uma turma, na ordem de chegada dos alunos.
- Guardar quais e-mails já confirmaram presença em um evento.

Questão 7. Implementação – Cadastro de participantes únicos. Crie um programa que:

- Declare um `ArrayList<String>` com uma lista de inscrições (incluindo nomes repetidos de propósito).
- Use um `HashSet<String>` para descobrir quantos participantes **únicos** se inscreveram.
- Imprima o total de inscrições e o total de participantes únicos.

Questão 8. Implementação – Catálogo de produtos. Crie um programa que:

- Use um `HashMap<String, Double>` para guardar pelo menos 4 produtos e seus preços.
- Permita “vender” um produto: aumente um contador de vendas (pode usar um segundo `HashMap<String, Integer>` para isso).
- Percorra o catálogo com `entrySet()` e imprima cada produto no formato: "Caderno: R\$ 12.50 (3 vendas)".
- Use um `ArrayList<String>` para registrar, em ordem, o nome de cada produto vendido durante a execução do programa.

Para revisar antes de implementar

- A estrutura escolhida precisa de **ordem**? Pense em `ArrayList`.
- A estrutura escolhida precisa impedir **repetição**? Pense em `HashSet`.
- A estrutura escolhida precisa **associar um dado a outro**? Pense em `HashMap`.
- É possível (e comum!) combinar as três no mesmo programa, cada uma resolvendo uma parte diferente do problema.